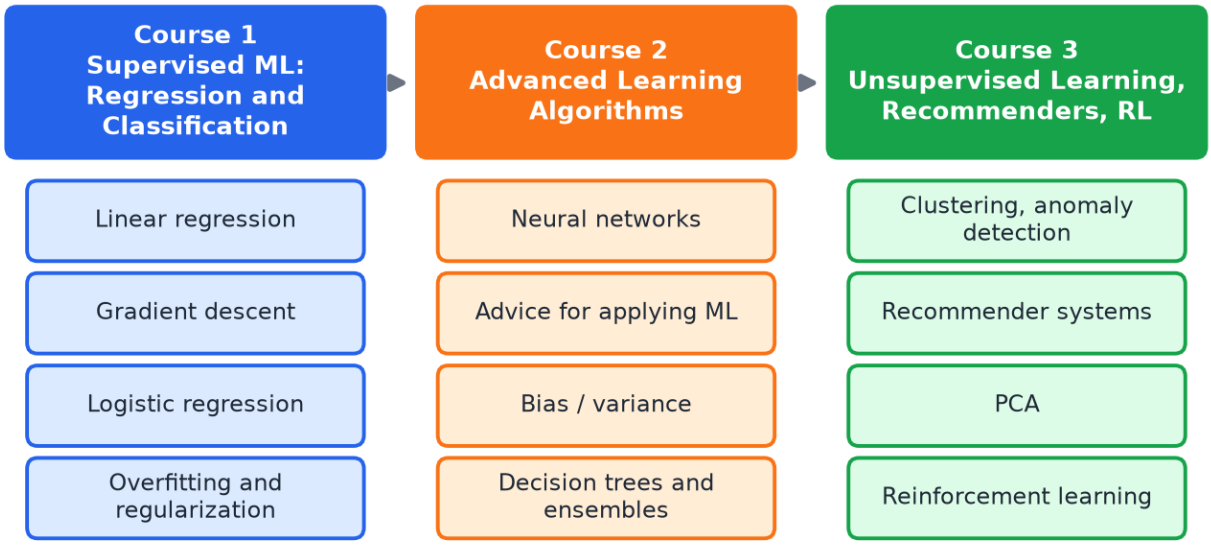
Machine Learning Specialization

Complete study notes for all three courses
Andrew Ng · DeepLearning.AI and Stanford Online



The three courses and the topics each one covers.

Written for revision: every concept has an explanation, the formula, a picture, and the common mistakes. These notes are based on the specialization's curriculum from memory, so check fine details such as exact numbers in examples against the lectures.

Contents

How to use these notes 3

Course 1: Supervised Machine Learning: Regression and Classification 4

1.1 What machine learning is, and the two big families . . . 4
1.2 Linear regression with one variable . . . 5
1.3 Gradient descent . . . 6
1.4 Multiple linear regression and feature scaling . . . 7
1.5 Classification and logistic regression . . . 8
1.6 Overfitting, underfitting and regularization . . . 10

Course 2: Advanced Learning Algorithms 12

2.1 Neural networks . . . 12
2.2 Advice for applying machine learning . . . 14
2.3 Decision trees and tree ensembles . . . 17

Course 3: Unsupervised Learning, Recommenders, Reinforcement Learning 19

3.1 Clustering with K-means . . . 19
3.2 Anomaly detection . . . 20
3.3 Recommender systems . . . 21
3.4 Principal component analysis (PCA) . . . 22
3.5 Reinforcement learning . . . 22

Cheat sheet and revision 25

Which algorithm for which job . . . 25
The universal workflow . . . 25
Common mistakes . . . 25
Applying this to your loan default project . . . 26
Revision questions . . . 26

How to use these notes

The specialization is three courses, each about four weeks. The flow is deliberate: Course 1 gives you the two basic models and the optimizer that trains them. Course 2 adds neural networks, the practical craft of diagnosing and improving models, and tree ensembles. Course 3 widens the toolbox to unsupervised learning, recommenders and reinforcement learning.

Each section follows the same pattern: **what problem is being solved**, **the idea**, **the maths**, **a picture**, and **what goes wrong in practice**. The coloured boxes mark key ideas, intuition, practical tips, and warnings. The last section has a one-page cheat sheet and revision questions.

Notation used throughout

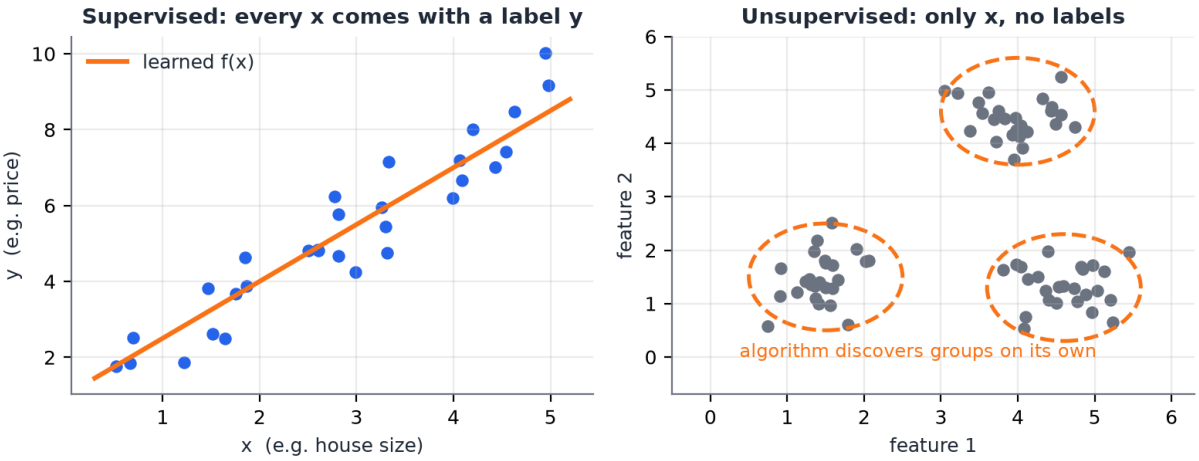
Symbol	Meaning
m	number of training examples
n	number of input features
$x^{(i)}$, $y^{(i)}$	the i-th training example: its input features and its true label (target)
$x_j^{(i)}$	value of feature j in training example i
w, b	model parameters: weights (one per feature) and a bias (a single number)
$f_{w,b}(x)$ or $\hat{y}$	the model's prediction for input x
J(w, b)	cost function: average loss over the whole training set; the thing we minimise
α	learning rate (step size of gradient descent)
λ	regularization strength
$a^{[l]}$	activation vector (outputs) of layer l in a neural network
K	number of clusters; also used for number of classes in softmax context
γ	discount factor in reinforcement learning

Course 1: Supervised Machine Learning: Regression and Classification

1.1 What machine learning is, and the two big families

Arthur Samuel's classic definition: machine learning is the field of study that gives computers the ability to learn without being explicitly programmed. Instead of writing rules by hand, we show a program many examples and let it find the rules. Almost everything in this specialization falls into one of three families.

Family	What you give the algorithm	What it learns	Typical uses
Supervised learning	Inputs x **with** correct outputs y	A mapping x → y that generalises to new x	Price prediction, spam filtering, loan default, medical diagnosis
Unsupervised learning	Inputs x only, no labels	Structure: groups, unusual points, compact representations	Customer segments, fraud / anomaly detection, visualisation
Reinforcement learning	A reward signal for actions in an environment	A policy: what action to take in each state	Game playing, robot control, scheduling



Left: supervised learning has a right answer for each example. Right: unsupervised learning has to discover the groups itself.

Regression versus classification

Both are supervised. **Regression** predicts a number from an infinite range of possible values (a house price, tomorrow's temperature). **Classification** predicts one of a small, fixed set of categories (spam or not spam, which digit 0 to 9, default or no default). Many classification models output a probability first and then apply a threshold.

Examples of unsupervised tasks

- **Clustering**: group similar items (news articles about the same story, customers with similar behaviour).
- **Anomaly detection**: find unusual events (fraudulent transactions, failing machines).

- **Dimensionality reduction**: compress many features into fewer while keeping most of the information.

1.2 Linear regression with one variable

Linear regression fits a straight line through the data. With a single input feature the model is:

$$f_{w,b}(x) = w \cdot x + b$$

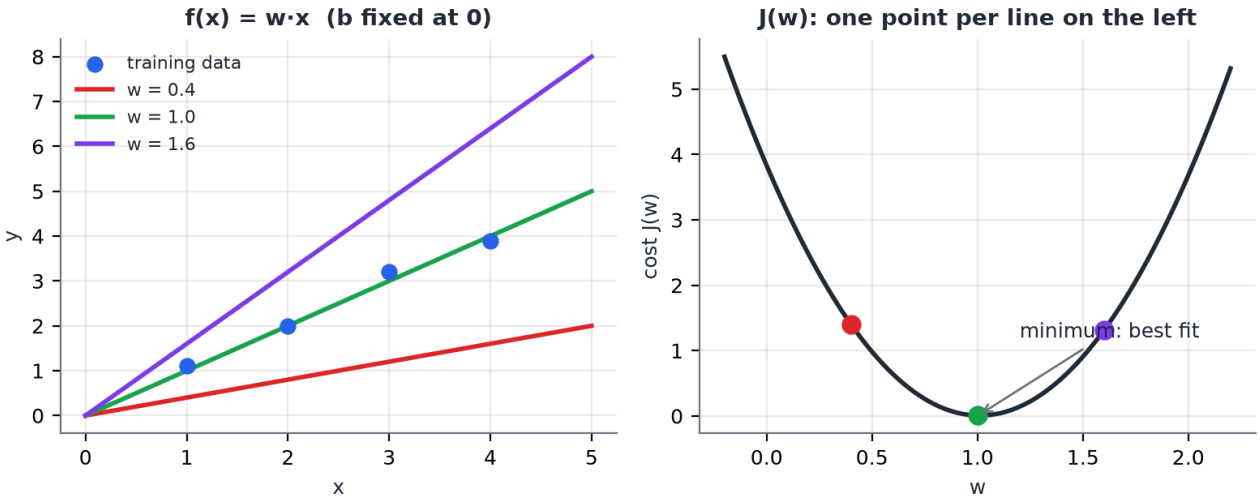
w is the slope and **b** is where the line crosses the y-axis. Training means choosing w and b so the line is as close as possible to all training points. To say what "close" means we define a cost function.

The cost function

For each example, the **error** is the prediction minus the true value, $\hat{y}^{(i)} - y^{(i)}$. Squaring it makes all errors positive and punishes big mistakes more. Averaging over the m examples gives:

$$J(w, b) = (1 / 2m) \cdot \Sigma_{i=1..m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2$$

The extra factor of 2 is only there so the derivative is tidier later. The goal of training is **minimise J over w and b**.



Left: three candidate lines. Right: each line is one point on the J(w) curve; the best line sits at the bottom of the bowl.

Intuition. Think of J as a landscape. Every choice of (w, b) is a location on the ground and J is the altitude. Training is walking downhill until you reach the lowest point. For linear regression with squared error the landscape is a smooth bowl with exactly one bottom (it is *convex*), so you cannot get stuck in a fake valley.

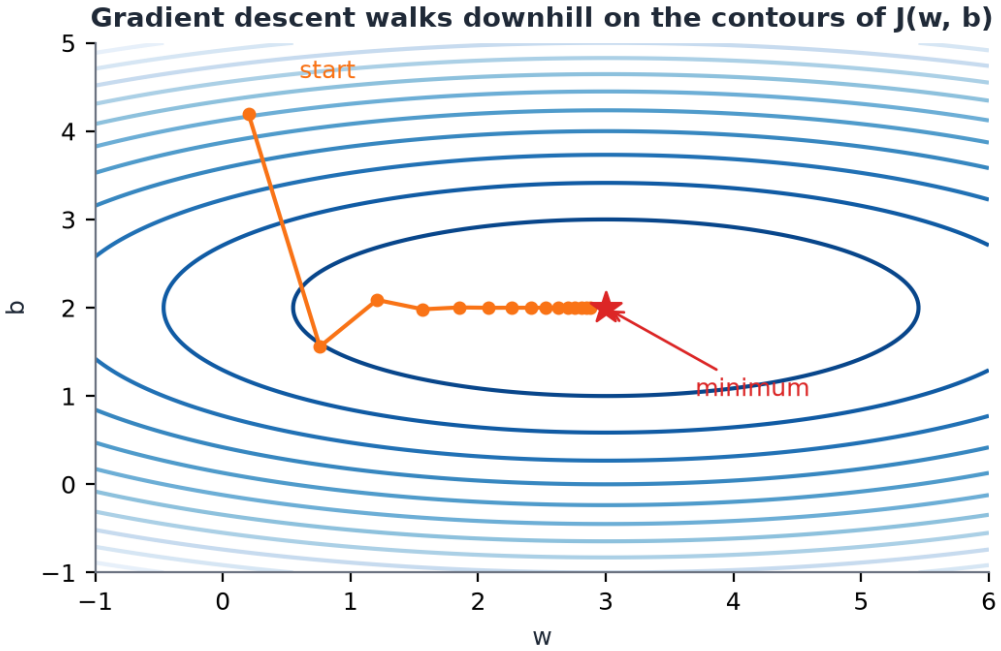
1.3 Gradient descent

Gradient descent is the general-purpose optimiser: start anywhere, look at the slope of J at your current position, and take a small step in the downhill direction. Repeat until the steps become tiny. The update rule is:

$$w := w - \alpha \cdot \partial J/\partial w \quad b := b - \alpha \cdot \partial J/\partial b$$

For linear regression the derivatives work out to:

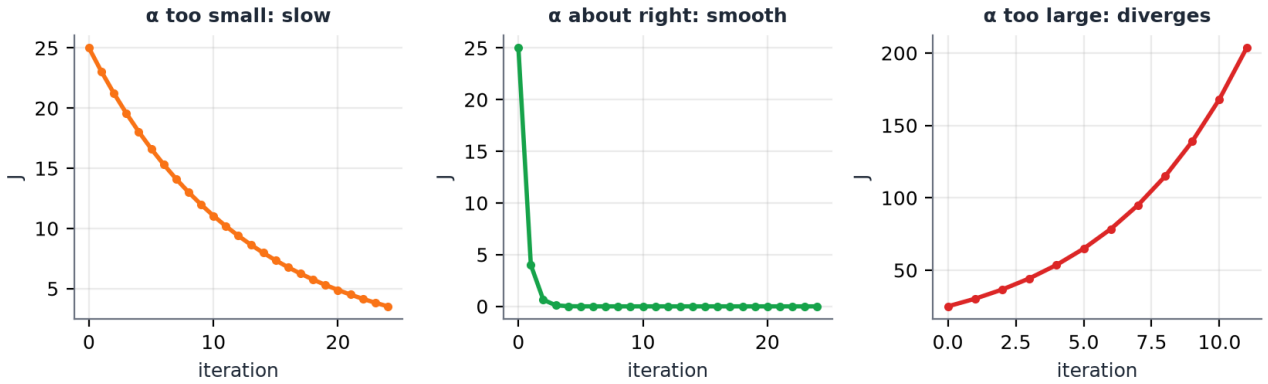
$$\partial J/\partial w = (1/m)\ \Sigma\ (\ f(x^{(i)}) - y^{(i)}\ ) \cdot x^{(i)} \quad \partial J/\partial b = (1/m)\ \Sigma\ (\ f(x^{(i)}) - y^{(i)}\ )$$



Gradient descent on a two-parameter cost. Each dot is one update; the path bends toward the minimum.

- **Simultaneous update**: compute both new values of w and b from the *old* w and b, then assign them together. Updating w first and then using the new w to compute b's step is a bug.
- **Batch** gradient descent uses all m training examples in every step. (Variants that use a subset exist; this is what the course implements.)
- **The slope sets the step**: near the minimum the slope is small so the steps shrink automatically, even with a fixed α.

Choosing the learning rate α



Plot J against iteration number. A healthy run decreases J on every iteration.

- **α too small**: it converges, but very slowly.
- **α too large**: steps overshoot the minimum, J bounces or even grows (diverges).
- **How to pick**: try values roughly 3× apart (0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1) and keep the largest one that still decreases J steadily.

Practical tip. If J ever *increases* from one iteration to the next, it is almost always either α too large or a bug in the gradient code. A good sanity test: set α to a tiny value, and J must decrease every step.

1.4 Multiple linear regression and feature scaling

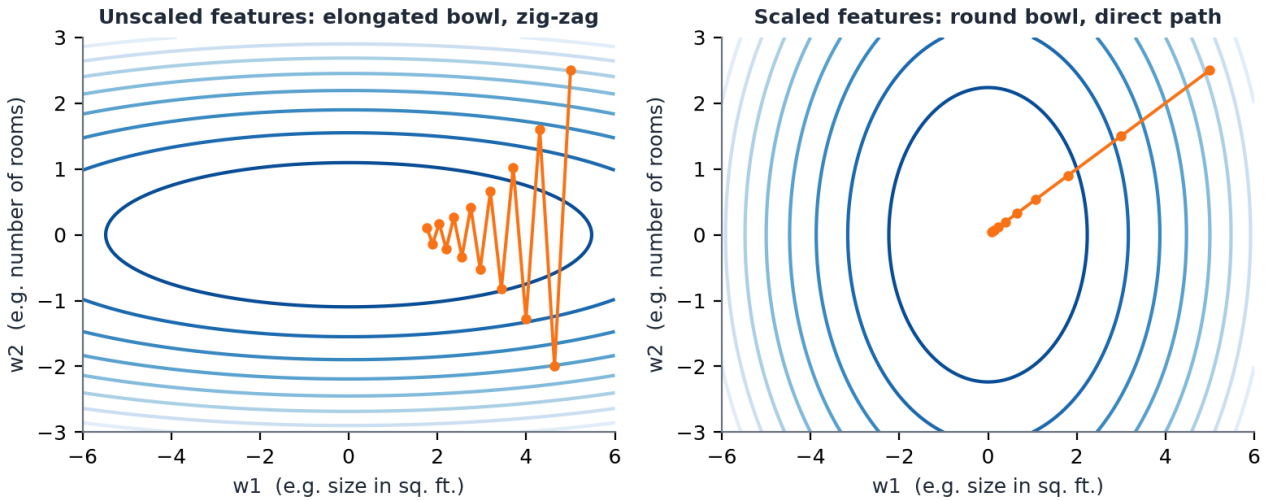
With n features the model is a weighted sum, written compactly as a dot product of two vectors:

$$f_{w,b}(x) = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b = w \cdot x + b$$

Gradient descent is the same idea; there is just one derivative per weight: $w_j := w_j - \alpha\,(1/m)\,\Sigma\,(f(x^{(i)}) - y^{(i)})\,x_j^{(i)}$. In code you write this with **vectorisation** (NumPy `np.dot`) instead of a Python loop. It is dramatically faster because it uses optimised, parallel hardware routines.

Why scale features

Suppose x_1 is house size (300 to 2000) and x_2 is number of bedrooms (0 to 5). A small change in w_1 changes the prediction a lot, while w_2 needs a big change to matter. The cost contours become a long thin ellipse and gradient descent zig-zags. Rescaling the features to similar ranges makes the contours round and the path direct.



Same problem, same α-style steps: scaling turns a slow zig-zag into a short, direct walk.

Method	Formula	Result
Divide by max	x / x_{max}	values in (0, 1]
Mean normalisation	$(x - \mu) / (x_{max} - x_{min})$	centred around 0, roughly −1 to 1
Z-score normalisation	$(x - \mu) / \sigma$	mean 0, standard deviation 1

Watch out. Compute μ and σ **on the training set only**, then reuse those same numbers to transform validation, test and future data. Fitting the scaler on all data leaks information from the test set.

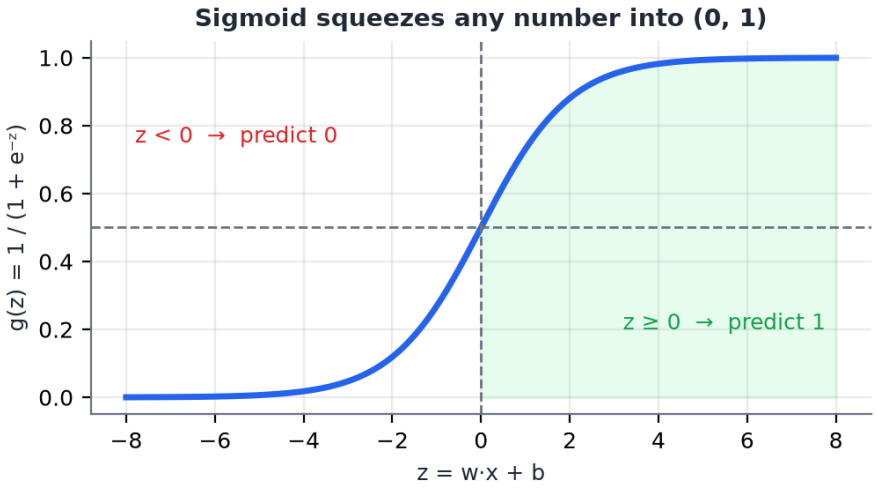
Feature engineering and polynomial regression

Feature engineering means using your knowledge of the problem to create better inputs. If a house has frontage and depth, their product (area) may predict price far better than either alone. Linear regression can also fit curves by feeding it powers of a feature: x, x^2, x^3 or $\sqrt{x}$. It is still "linear" because it is linear in the *weights*. When you add polynomial features, scaling becomes essential because x^3 is enormous compared with x.

1.5 Classification and logistic regression

Predicting a category with linear regression fails: a single far-away example shifts the line and moves the 0.5 threshold, flipping predictions for many points. Logistic regression is designed for this. It first computes the same linear score $z = w \cdot x + b$, then passes it through the **sigmoid** function so the output is a number between 0 and 1.

$$g(z) = 1 / (1 + e^{-z}) \quad f_{w,b}(x) = g(w \cdot x + b)$$

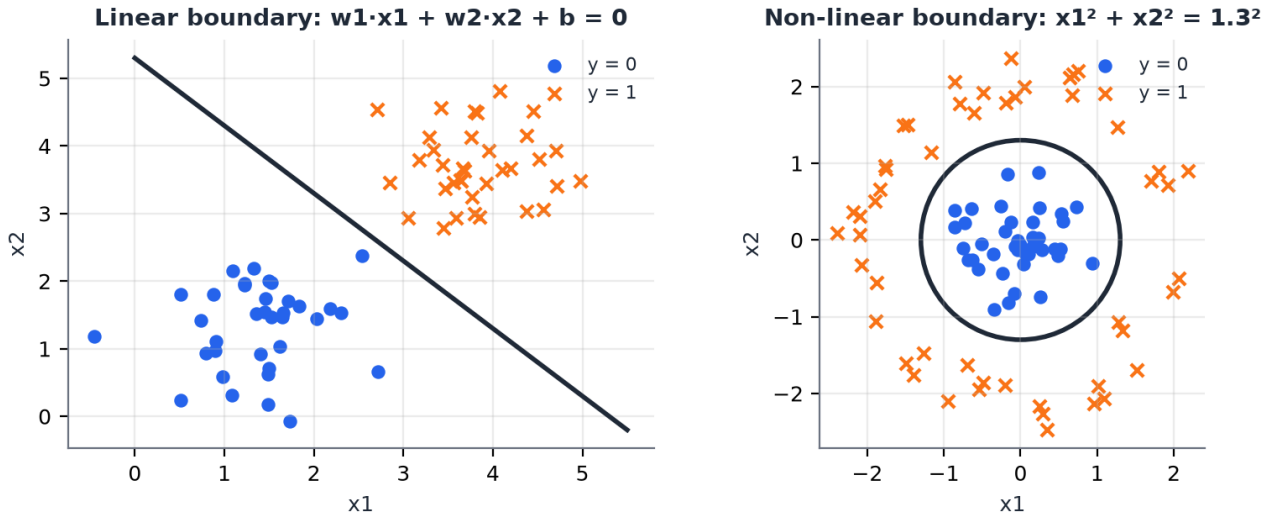


The sigmoid: large positive z gives near 1, large negative z gives near 0, and z = 0 gives exactly 0.5.

Read the output as **the probability that y = 1 given x**. To get a hard decision apply a threshold, usually 0.5: predict 1 when f ≥ 0.5, which happens exactly when z = w · x + b ≥ 0.

Decision boundary

The set of points where z = 0 is the **decision boundary**. With two features and plain linear z it is a straight line. Add polynomial features (x_1^2, x_2^2, x_1x_2) and the same model draws circles, ellipses and more complex shapes.



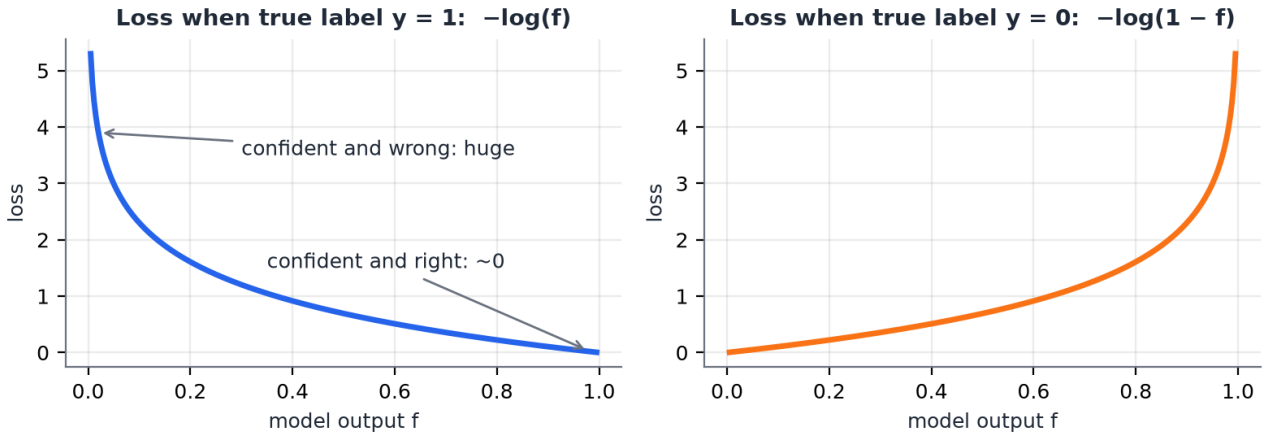
Left: a linear decision boundary. Right: with squared features the boundary becomes a circle.

Loss for logistic regression

Squared error with a sigmoid creates a bumpy, non-convex cost with many local minima. Instead we use **log loss** (binary cross-entropy), which is derived from maximum likelihood and is convex:

$$L(f, y) = -y \cdot \log(f) - (1 - y) \cdot \log(1 - f)$$

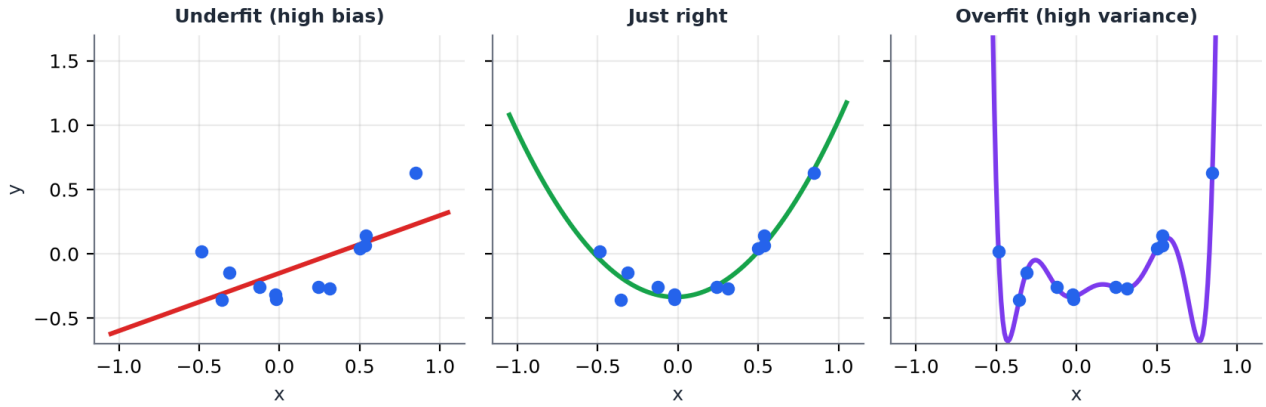
Because y is either 0 or 1, only one of the two terms is ever active. If y = 1 the loss is −log(f); if y = 0 it is −log(1 − f). The cost J is the average loss over all examples.



The loss is near zero when the model is confidently right and explodes when it is confidently wrong.

The gradient descent updates for logistic regression look *identical* to linear regression: $w_j := w_j - \alpha\ (1/m)\ \Sigma\ (\ f(x^{(i)}) - y^{(i)}\ )\ x_j^{(i)}$. The only difference is that f is now the sigmoid of $w \cdot x + b$.

1.6 Overfitting, underfitting and regularization



The same data fitted by a line, a quadratic and a degree-9 polynomial.

Term	Also called	What it looks like	Typical fix
Underfitting	High bias	Poor on training *and* new data	More features, more complex model, less regularization
Good fit	Generalises	Good on training, almost as good on new data	—
Overfitting	High variance	Excellent on training, poor on new data	More data, fewer features, regularization

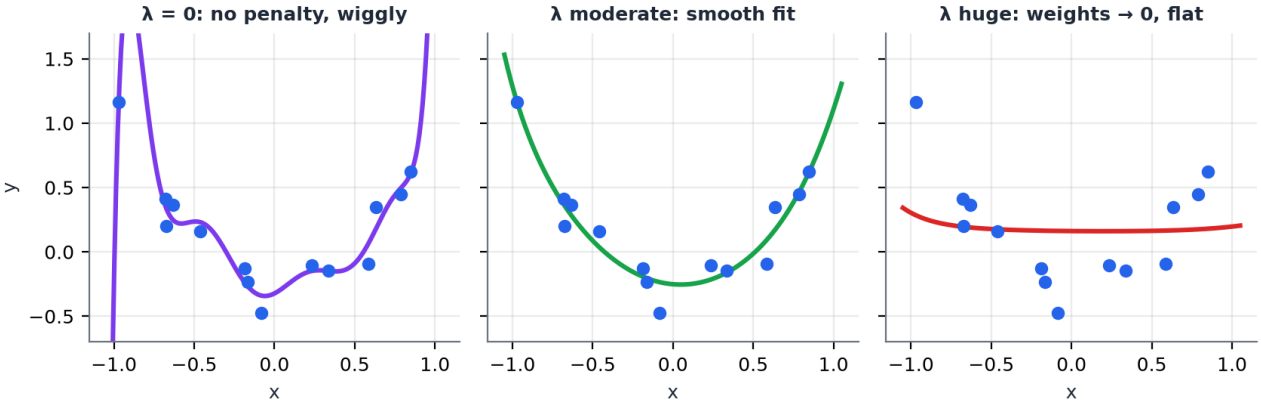
Three ways to fight overfitting

- **Collect more training data**: the most reliable cure, when it is possible.
- **Select fewer features**: drop features that carry little information (the cost is losing some useful signal).
- **Regularization**: keep all features but shrink the weights so no single one dominates.

How regularization works

Add a penalty on the size of the weights to the cost. The model must now trade off fitting the data against keeping weights small:

$$J_{reg}(w, b) = (1/2m)\ \Sigma\ (\ f(x^{(i)}) - y^{(i)}\ )^2 + (\lambda\ /\ 2m)\ \Sigma_{j=1..n}\ w_j^{\ 2}$$



A degree-9 polynomial under different λ. The best λ sits between wiggly and flat.

- **λ = 0**: no penalty, free to overfit.
- **λ moderate**: large weights are discouraged, the curve smooths out.
- **λ huge**: every weight is pushed to nearly 0, the model becomes the flat line f = b (underfit).
- The bias b is conventionally **not** regularized; it makes little difference either way.

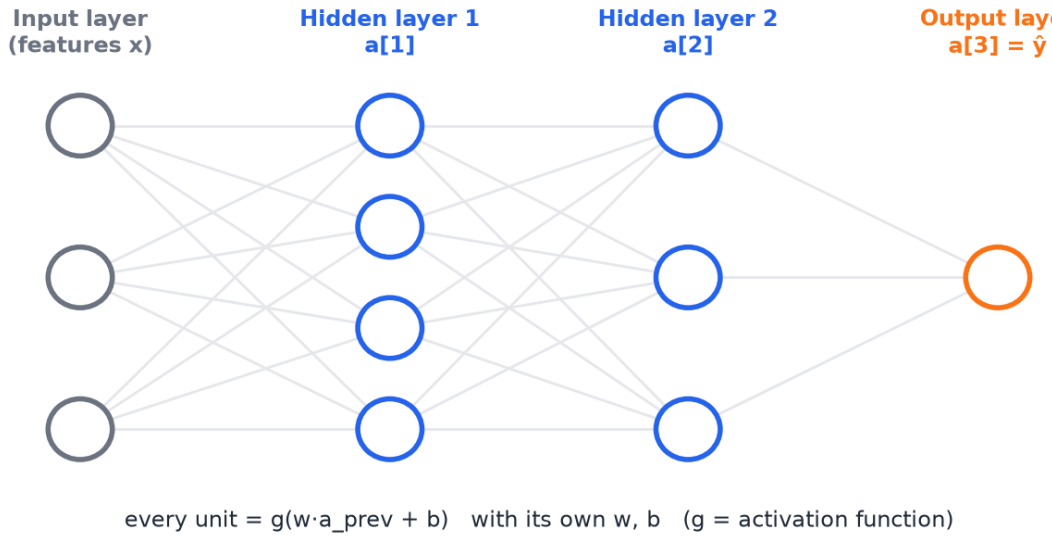
Intuition. In the gradient descent step the weight update becomes $w_j := w_j \cdot (1 - \alpha\lambda/m) - \alpha \cdot$(usual gradient). Each iteration first *shrinks* w_j slightly, then applies the normal correction. That constant gentle shrinkage is what keeps weights small.

Practical tip. Regularization works the same way for logistic regression: add the identical $(\lambda/2m)\ \Sigma\ w^2$ term to the log-loss cost. Scale your features first, otherwise the penalty treats features unfairly.

Course 2: Advanced Learning Algorithms

2.1 Neural networks

A neural network is many logistic-regression-like units arranged in layers. Each unit takes the previous layer's outputs, forms a weighted sum plus bias, and applies an activation function g. Early layers learn simple features; later layers combine them into richer ones. The big benefit is that you do **not** hand-craft features: the hidden layers learn them from data.



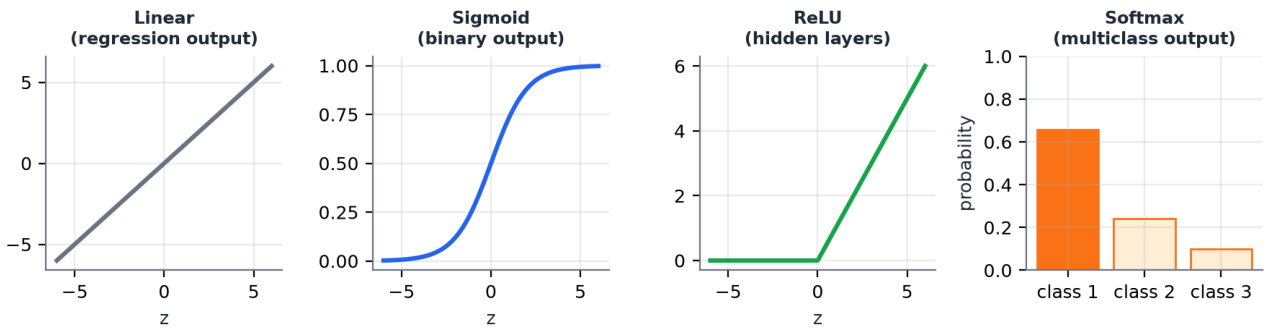
A 3-layer network (not counting the input layer). Every arrow carries a weight.

What one layer computes

$$a_j^{[l]} = g\left(w_j^{[l]} \cdot a^{[l-1]} + b_j^{[l]} \right)$$

Read it as: unit j in layer l has its own weight vector and bias, looks at all activations from layer l−1 (layer 0 is the input x), and outputs one number. Stack the units of a layer and you get a vector $a^{[l]}$. **Forward propagation** is just doing this layer by layer from input to output to produce a prediction.

Choosing activation functions



The activation functions you need. Hidden layers: ReLU by default. Output layer: pick by task.

Where	Task	Activation	Reason
Output layer	Binary classification	Sigmoid	Output is a probability in (0, 1)
Output layer	Regression, any sign	Linear	Output can be any real number
Output layer	Regression, never negative	ReLU	Output is ≥ 0 (e.g. price)
Output layer	Multiclass classification	Softmax	Probabilities over K classes summing to 1
Hidden layers	All tasks	ReLU (default)	Cheap and avoids slow-learning flat regions

Watch out. If you used a linear activation in every hidden layer, the whole network would collapse into one big linear regression, no matter how many layers. The non-linearity is what gives depth its power.

Multiclass classification with softmax

For K classes the output layer has K units that produce scores $z_1 \dots z_K$. Softmax turns them into probabilities:

$$a_j = e^{z_j} / (e^{z_1} + \dots + e^{z_K})$$

The probabilities always sum to 1. The loss is the negative log of the probability given to the true class, which is the multiclass version of log loss. **Multi-label** problems (an image can contain a car *and* a person) are different: use independent sigmoid outputs, one per label, not softmax.

Practical tip. For numerical stability, let the output layer be linear and tell the loss function to apply softmax or sigmoid internally (in TensorFlow: `from_logits=True`). It avoids round-off errors from computing e^z with large z.

Training a network: the three steps

- **Specify the model**: layers, units per layer, activations.
- **Specify the loss and cost**: binary cross-entropy for classification, mean squared error for regression, categorical cross-entropy for multiclass.
- **Train**: gradient descent on all the weights. Gradients come from **backpropagation**, which applies the chain rule efficiently through the computation graph.

In practice the **Adam** optimiser is the default instead of plain gradient descent. It keeps a separate, automatically adapting learning rate for every parameter: it raises the rate when it keeps moving in the same direction and lowers it when it oscillates. It is far less sensitive to the initial α.

Beyond **dense** layers (each unit sees all inputs), there are **convolutional** layers where each unit looks at only a patch of the input. Fewer parameters, faster computation, and less overfitting; common for images and time series.

2.2 Advice for applying machine learning

This is the most practically valuable part of the specialization. A model that scores poorly could be failing for very different reasons, and the right fix depends on which. The methods below tell you *where to spend your time*.

Train / cross-validation / test split

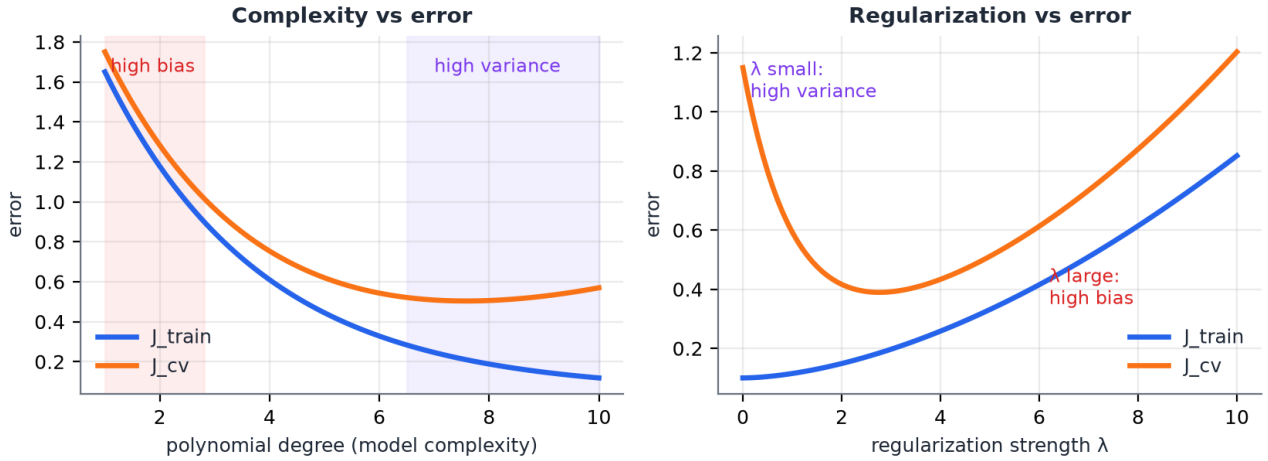


Three data splits with three different jobs.

- **Training set** fits the parameters w, b.
- **Cross-validation (dev) set** is used to choose between models: polynomial degree, λ, network size.
- **Test set** is touched **once**, at the very end, to report an unbiased estimate of real-world performance.

Why not just use the test set to choose? Because the more decisions you make by looking at it, the more it becomes part of training, and the estimate becomes optimistic. Cost on each split is computed **without** the regularization term: J_{train}, J_{cv}, J_{test} measure plain error.

Diagnosing bias and variance



Left: as the model gets more complex, J_train keeps falling but J_cv turns back up. Right: λ moves you along the same axis in the opposite direction.

Diagnosis	J_train	J_cv	Meaning
High bias (underfit)	High	High (close to J_train)	Model too simple, cannot even fit the training data
High variance (overfit)	Low	Much higher than J_train	Model memorised the training data
Both	High	Much higher	Happens in neural networks with some regions overfit

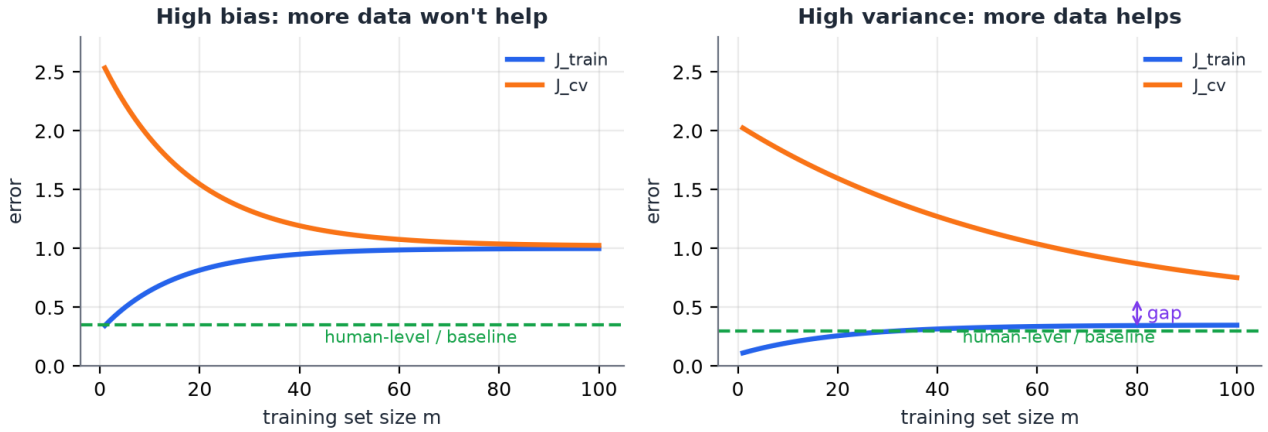
Diagnosis	J_train	J_cv	Meaning
Good	Low	Low	Done, or at the data's noise floor

What counts as "high"? Use a baseline

An error of 10.8% sounds bad until you learn that human experts get 10.6% on the same task. Always compare against a **baseline**: human-level performance, a competing algorithm, or a simple guess. The two gaps that matter are:

- **Baseline − J_{train}**: a large gap means high bias.
- **J_{cv} − J_{train}**: a large gap means high variance.

Learning curves



Learning curves show error as a function of training set size.

With **high bias** both curves flatten at a high error; adding data will not help and you need a more expressive model. With **high variance** there is a large gap that narrows as data grows, so more data (or less complexity) helps.

What to try next

Symptom	Things to try
High variance	More training data; fewer features; increase λ; simplify the model
High bias	Add features or polynomial terms; decrease λ; bigger / deeper network
Neural networks	Rule of thumb: a large network trained with good regularization is usually low-bias; fix high bias by enlarging, then high variance by adding data or regularizing. Bigger is rarely harmful if regularized.

Iterative development and error analysis

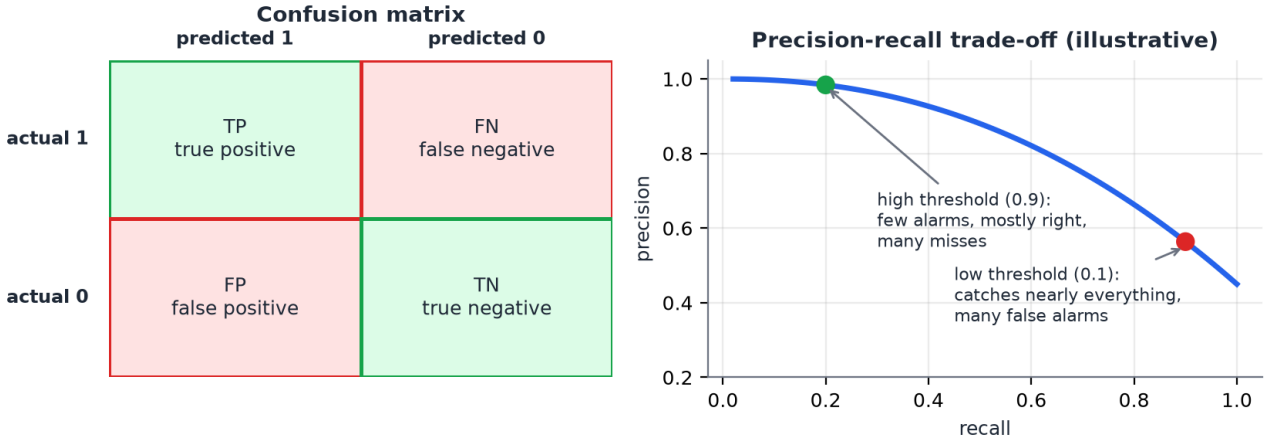
Machine learning work is a loop: choose architecture and data, train, diagnose (bias/variance and **error analysis**), adjust, repeat. For error analysis, hand-examine a sample of misclassified validation examples, tag them by cause (for example "pharma spam", "misspelled words", "phishing links"), and count. Fix the largest category first. It prevents you from spending a month on a problem that affects 2% of errors.

Adding data, augmentation and transfer learning

- **Targeted data collection**: collect more of the kind of examples the error analysis showed to be hard.
- **Data augmentation**: create new examples by transforming real ones (rotate, crop, add noise to audio). The transformation must keep the label correct.
- **Data synthesis**: generate artificial examples (render text with different fonts).
- **Transfer learning**: take a network pretrained on a large dataset, replace the output layer, and either train only the new layer (small data) or fine-tune all layers (more data). The early layers' features (edges, shapes) transfer well.

Skewed datasets: precision, recall and F1

When 99.5% of examples are negative, a model that always says "negative" has 99.5% accuracy and is useless. Use the confusion matrix instead.



Left: the four outcomes. Right: moving the decision threshold slides you along the precision-recall trade-off.

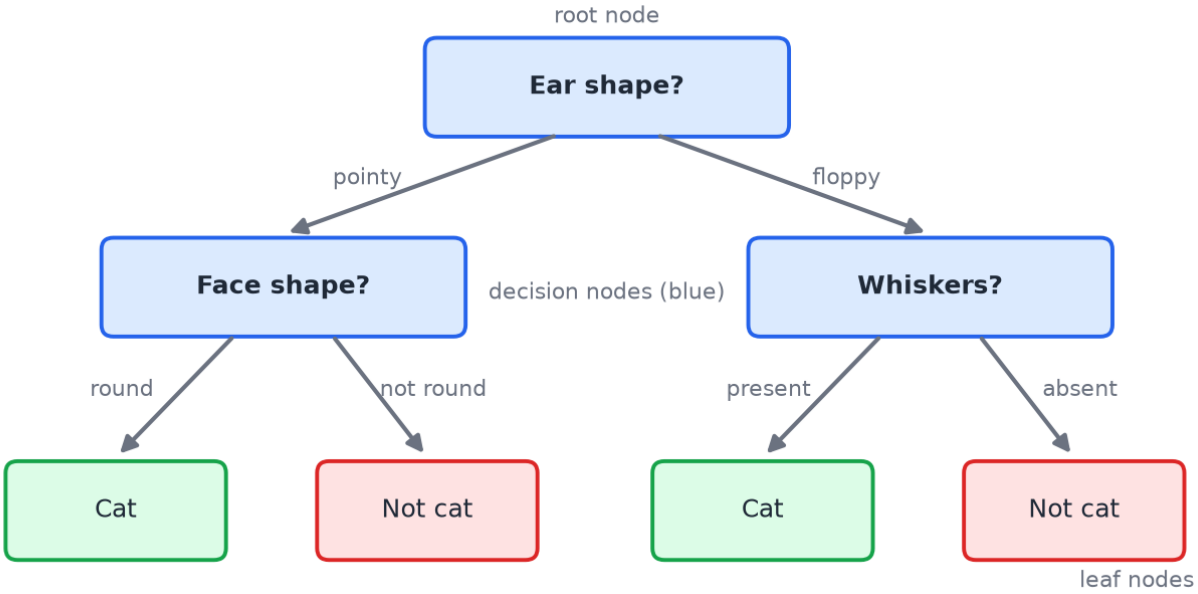
$$\text{Precision} = TP / (TP + FP) \quad \text{Recall} = TP / (TP + FN)$$

- **Precision**: of everything I flagged as positive, how much really is positive? Matters when false alarms are costly.
- **Recall**: of all the real positives, how many did I find? Matters when missing one is costly (disease, fraud).
- **F1 score** combines them with the harmonic mean: $F1 = 2PR / (P + R)$. It is low unless *both* are reasonable, so one cannot hide behind the other.

Practical tip. To trade precision for recall just change the threshold. Raising it above 0.5 makes the model more certain before it says yes (precision up, recall down). Lowering it does the opposite. Choose the threshold by the real-world cost of each kind of mistake, using the validation set.

2.3 Decision trees and tree ensembles

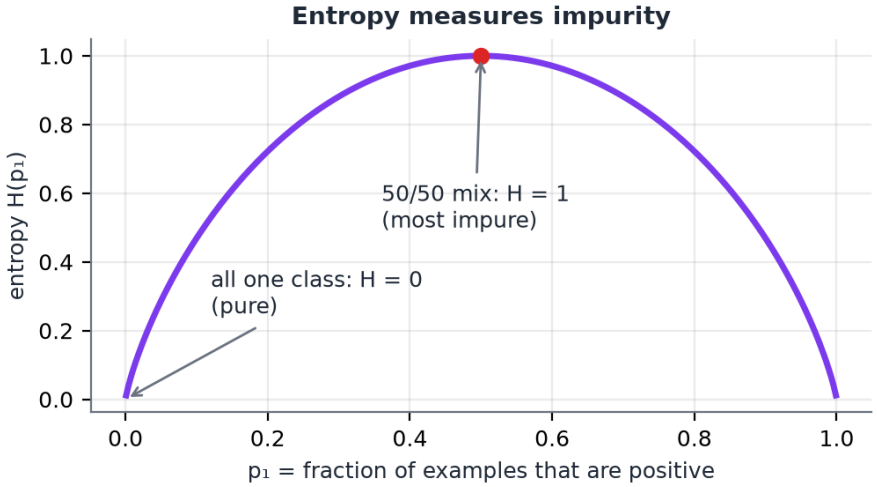
A decision tree asks a sequence of yes/no questions about features, following branches until it reaches a leaf that holds a prediction. Trees learn non-linear rules naturally and are easy to explain.



A tree that classifies cat vs not cat from categorical features.

How the tree is learned

- **Step 1**: at the current node, try every available feature as the split.
- **Step 2**: choose the split that gives the highest **information gain**, i.e. the largest drop in impurity.
- **Step 3**: recurse on each branch. Stop when a node is pure, the depth limit is reached, the gain is too small, or too few examples remain.



Entropy is 0 for a pure node and 1 for a 50/50 mix.

$$H(p_1) = -p_1 \log_2(p_1) - (1 - p_1) \log_2(1 - p_1)$$

$$\text{Information gain} = H(p_{root}) - \left(w_{left} \cdot H(p_{left}) + w_{right} \cdot H(p_{right}) \right)$$

Here w_{left} and w_{right} are the fractions of the node's examples that go left and right. Weighting matters: a split that isolates 2 examples perfectly should not outweigh one that cleans up 50.

Details that come up in practice

- **Many-valued categorical features**: use **one-hot encoding**, one binary column per category.
- **Continuous features**: try thresholds (e.g. weight ≤ 9 lbs) between sorted values and take the best by information gain.
- **Regression trees**: leaf predicts the mean of its examples; the split criterion is the reduction in **variance** instead of entropy.
- **Instability**: a tiny change in the data can change the whole tree, which motivates ensembles.

Ensembles

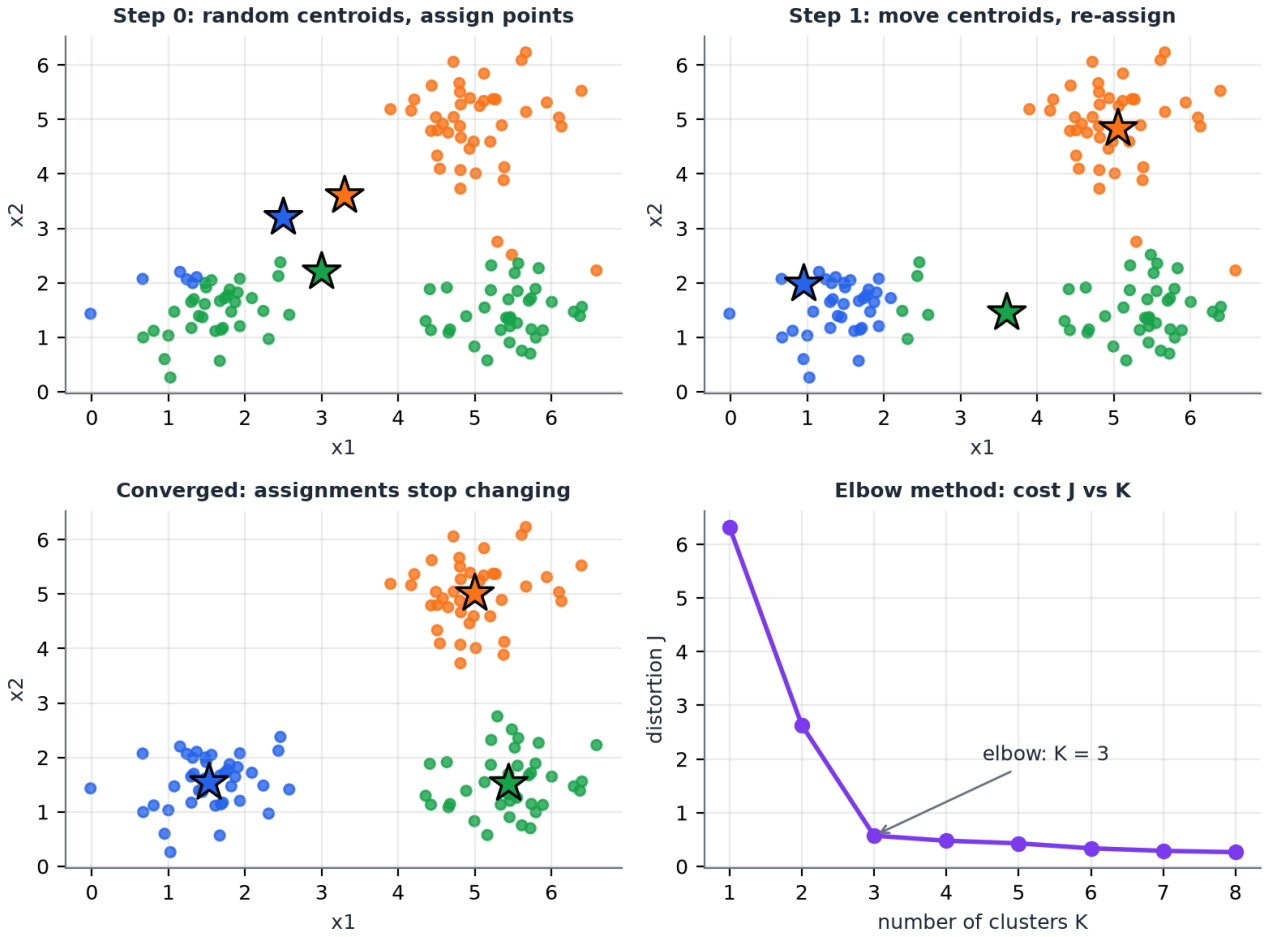
- **Bagging / random forest**: train B trees (often tens to over a hundred), each on a random sample of the data drawn *with replacement* (bootstrap). At each split also consider only a random subset of about √n features. Predict by majority vote. Averaging many noisy trees reduces variance.
- **Boosted trees (XGBoost)**: build trees one after another; each new tree concentrates more on the examples that previous trees got wrong ("deliberate practice"). Implementations add regularization and built-in handling of missing values and are a very strong default for tabular data.

	Decision trees / ensembles	Neural networks
Best data type	Tabular, structured data	Images, audio, text, and also tabular
Training speed	Fast	Can be slow
Interpretability	Single small trees are readable	Hard to inspect
Pre-processing	Little; no scaling needed	Scaling usually needed
Transfer learning	No	Yes
Chaining models	Awkward	Easy: several networks train together

Course 3: Unsupervised Learning, Recommenders, Reinforcement Learning

3.1 Clustering with K-means

Clustering groups unlabeled examples so that members of a group are more similar to each other than to other groups. K-means, the workhorse algorithm, is two repeated steps.



K-means in action (top row and bottom left) and the elbow method for choosing K (bottom right).

- **Initialise**: choose K starting centroids, usually K random training examples.
- **Assign**: give each point to its nearest centroid (Euclidean distance).
- **Update**: move each centroid to the mean of the points assigned to it.
- Repeat assign + update until the assignments stop changing. The algorithm is guaranteed to converge.

$$J = (1/m)\ \Sigma_i\ ||\ x^{(i)} - \mu_{c(i)}\ ||^2 \text{ (the distortion cost)}$$

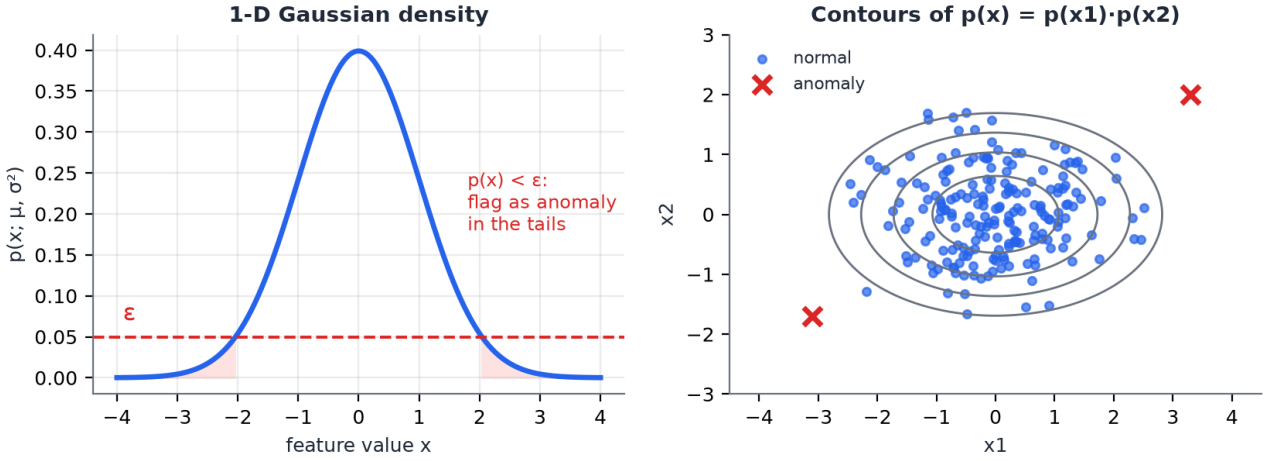
J never increases from one iteration to the next; if it does, there is a bug. The final result depends on the starting centroids and may be a poor local optimum, so **run K-means 50 to 1000 times with different random initialisations and keep the run with the lowest J**.

Choosing K

The **elbow method** plots J against K and looks for a bend after which extra clusters give little improvement. Often there is no clear elbow. The more reliable approach: decide by the **downstream purpose** (for t-shirt sizes do you want three sizes or five?). K-means is an exploratory tool and there is rarely one "correct" K.

3.2 Anomaly detection

Given many normal examples, flag new ones that look unlike them. Useful for fraud, manufacturing defects, or machines about to fail. The approach is **density estimation**: model p(x), the probability of a normal example, and flag x as anomalous when p(x) < ε.



Left: a Gaussian density with a threshold ε. Right: in two dimensions, the contours of p(x) and points that fall in low-density regions.

$$p(x) = \Pi_{j=1..n}\, p(\, x_j \,;\, \mu_j,\, \sigma_j^{\,2}\,)$$

Fit μ_j and σ_j^2 for each feature as the mean and variance over the training set. The product assumes features are independent, which is wrong but works well in practice.

Choosing ε and the features

- Keep a **small number of labelled anomalies** in the cross-validation and test sets (training stays unlabeled, essentially all normal). Choose ε that maximises F1 on the cross-validation set.
- Features should look roughly Gaussian. Transform skewed ones with log(x), √x, or log(x + c).
- If an anomaly is not unusual in any single feature but unusual in a **combination** (CPU load high while network traffic is low), add a combined feature such as the ratio of the two.

Use anomaly detection when	Use supervised classification when
Very few positives (0 to 20) and many types of anomaly	Plenty of positive and negative examples
Future anomalies may look nothing like past ones (new fraud methods)	Future positives will resemble the training positives (spam)

3.3 Recommender systems

Collaborative filtering

You have a matrix of ratings: rows are movies, columns are users, many cells empty. The goal is to predict the empty cells. Each movie i gets a feature vector $x^{(i)}$ and each user j gets parameters $w^{(j)}$, $b^{(j)}$. A predicted rating is $w^{(j)} \cdot x^{(i)} + b^{(j)}$.

Movie	Alice	Bob	Carol	Dave
Love at Last	5	5	0	0
Romance Forever	5	?	?	0
Cute Puppies	?	4	0	?
Nonstop Car Chases	0	0	5	4
Swords vs Karate	0	0	5	?

The clever part: if the features $x^{(i)}$ are *unknown* too, learn them at the same time, because users who rated similar movies give information about what features matter. The cost function includes both sets of parameters, summing squared error only over the cells that were actually rated (r(i, j) = 1):

$$J = \tfrac{1}{2} \sum_{(i,j):\ r(i,j)=1} \left(w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \text{regularization on w and x}$$

- Minimise J with gradient descent over w, b **and** x together.
- **Binary labels** (liked / not liked, clicked / not clicked): use a sigmoid and log loss, as in logistic regression.
- **Mean normalisation**: subtract each movie's average rating before training and add it back at prediction. A brand-new user with no ratings then gets sensible average predictions rather than zeros.
- **Related items**: movies whose learned vectors $x^{(i)}$ are close (small squared distance) are similar.

Watch out. Collaborative filtering has the **cold-start problem**: new items and new users have no ratings. It also cannot use side information such as genre or age directly. Content-based filtering fixes both.

Content-based filtering

Instead of only using rating patterns, use **features of users and of items** and learn to match them. Build two neural networks, a **user network** and an **item network**, each producing a vector of the same length (e.g. 32 numbers) from raw features. Their dot product is the predicted affinity. Because both networks have the same output size, this is called a **two-tower** architecture.

Retrieval and ranking at scale

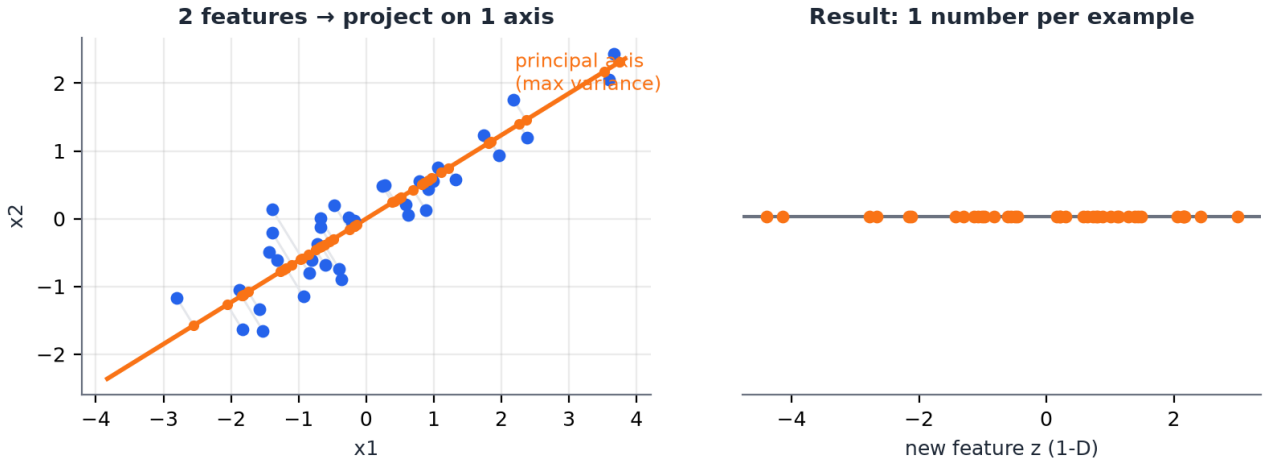
- **Retrieval**: quickly narrow millions of items to a few hundred candidates (items similar to recently watched, popular in the user's country). Item vectors can be precomputed offline.
- **Ranking**: score only those candidates with the full model and show them in order.

- More candidates improves recommendation quality but slows response; tune this trade-off with offline experiments.

Ethical note from the course: recommenders optimise a proxy (engagement, clicks) that can diverge from user wellbeing. Be deliberate about the objective, filter harmful items, and be transparent.

3.4 Principal component analysis (PCA)

PCA reduces the number of features while preserving as much of the variation as possible. In the course it is used mainly for **visualisation**: squeeze 50 features to 2 or 3 so you can plot the data and look for structure.



Each point is projected onto the axis along which the data varies most. Two numbers become one.

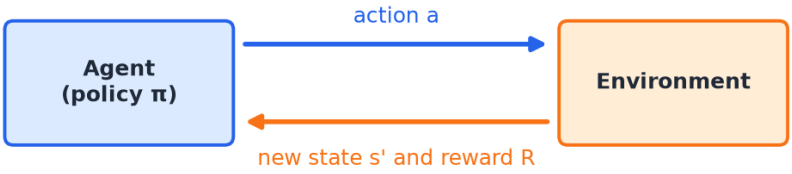
- **Preprocess**: scale features and subtract the mean.
- **Find the principal axis**: the direction in which projected points spread out most (maximum variance), so as little information as possible is lost.
- **Next axis**: perpendicular to the first, again maximum remaining variance, and so on.
- **Project** each example onto the chosen axes. The coordinates along those axes are the new features.
- **Reconstruction**: you can approximately go back, with the error equal to what was thrown away.

> **Watch out.** PCA is **not** linear regression. Regression minimises vertical distances to predict a special target y, while PCA minimises perpendicular distances and treats all features equally with no target.

3.5 Reinforcement learning

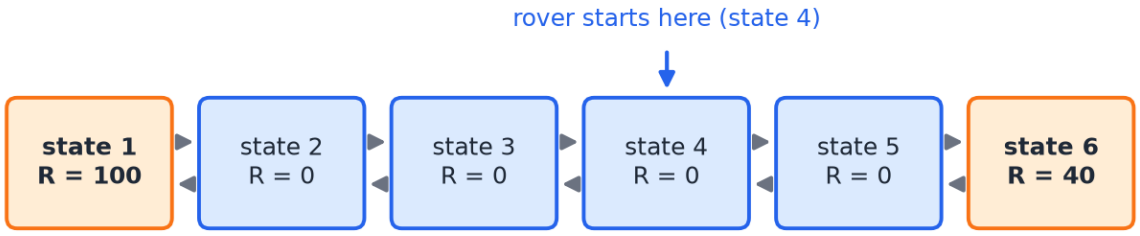
Instead of labelled examples, an agent interacts with an environment, taking actions and receiving **rewards** that tell it how well it is doing, but not what the right action was. The reward is the specification of the goal; the algorithm must work out how to achieve it.

goal: choose actions that maximise the total discounted return



The reinforcement learning loop.

Term	Meaning
State s	The situation the agent is in (position, speed, board layout)
Action a	A choice the agent can make
Reward R(s)	Immediate feedback for being in a state
Return	$R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$; total reward, with later rewards discounted
Discount factor γ	Number a bit below 1 (e.g. 0.9 or 0.99). Makes the agent prefer rewards sooner.
Policy π(s)	The rule that maps a state to an action; this is what we are trying to learn
Markov Decision Process	The formal model: the future depends only on the current state and action, not the history



γ = 0.5 | go left from 4: 0 + 0.5·0 + 0.25·0 + 0.125·100 = 12.5 | go right: 0 + 0.5·0 + 0.25·40 = 10

→ best first move from state 4 is LEFT (return 12.5 > 10)

The Mars rover example: from state 4, going left returns 12.5, going right returns 10, with γ = 0.5.

The state-action value function Q(s, a)

Q(s, a) is the return you get if you start in state s, take action a once, and behave optimally afterwards. If you know Q for all actions, the best policy is trivial: in every state pick the action with the largest Q.

$$Q(s, a) = R(s) + \gamma \cdot \max_{a'} Q(s', a')$$

This is the **Bellman equation**. It says that the value of an action is the immediate reward plus the discounted value of the best thing you can do from wherever you land (s'). It breaks a

long-horizon problem into one-step relationships, which is what makes learning feasible.

Continuous states and Deep Q-Networks

Real problems (a lunar lander with x, y, velocity, angle...) have continuous states, so a Q table is impossible. Use a neural network to **approximate Q**:

- Network input: the state s (and the action, or one output per action). Output: Q values for each action.
- Generate training data by playing: each experience tuple is (s, a, R(s), s'). The target is $y = R(s) + \gamma \max_{a'} Q(s', a')$. Train the network to match it as a normal regression.
- **ε-greedy exploration**: with probability $1 - \varepsilon$ choose the action with the largest Q (exploit); with probability ε choose randomly (explore). Start with ε high (e.g. 1.0) and decay to about 0.01.
- **Experience replay**: store recent tuples in a buffer and train on random **mini-batches** from it. Faster per step and breaks correlation between consecutive samples.
- **Soft updates**: update the target network slowly (new = 0.01 · new + 0.99 · old) so training does not oscillate or diverge.

> **Key idea.** Reinforcement learning is much less mature than supervised learning, and is harder to tune. In simulation it works impressively; in the real world it is rarer. Know the vocabulary and the Bellman equation, as they appear again in many areas.

Cheat sheet and revision

Which algorithm for which job

Task	First thing to try	Key knobs	Watch for
Predict a number, roughly linear	Linear regression	Features, λ, α	Scaling, outliers
Binary yes/no with probabilities	Logistic regression	λ, threshold	Imbalanced classes, calibration
Tabular, non-linear	Gradient-boosted trees (XGBoost)	Depth, # trees, learning rate	Overfitting, leakage
Images, audio, text	Neural network (pretrained)	Architecture, Adam, λ	Data size, overfitting
Group unlabeled data	K-means	K, # initialisations	Scale of features, local minima
Rare unusual events	Gaussian anomaly detection	ε, feature transforms	Non-Gaussian features
Recommend items	Collaborative / content-based	Embedding size, λ	Cold start
Visualise many features	PCA	Number of components	Scale features first
Sequential decisions with rewards	Q-learning / DQN	γ, ε, buffer size	Instability, reward design

The universal workflow

- **1. Define the goal and metric**, with the real cost of each kind of error.
- **2. Split the data first** (train / validation / test; stratify for imbalanced classes; split by time if data is temporal).
- **3. Start simple**: a baseline model you can beat, and a human or heuristic benchmark.
- **4. Diagnose**: compare J_{train} and J_{cv} with the baseline to find bias or variance.
- **5. Improve** with the matching fix, change one thing at a time.
- **6. Error analysis** on misclassified validation examples.
- **7. Evaluate once on the test set**, then deploy and monitor for drift.

Common mistakes

- Fitting scalers, imputers or feature selection on all the data before splitting (data leakage).
- Reporting accuracy on a skewed dataset instead of precision, recall, F1 or PR-AUC.
- Tuning on the test set until the number looks good.
- Regularizing without scaling the features.
- Reading a high training error as "need more data" when it actually means "need a more expressive model".

Applying this to your loan default project

Several ideas from the specialization map directly onto a default-prediction problem. They are pointers, not a prescription:

- **Class imbalance**: defaults are usually the minority. Accuracy is misleading; look at the confusion matrix, precision, recall and F1, and think about which error costs the lender more.
- **Threshold is a business decision**: the 0.5 cut-off is arbitrary. Choose it on validation data given the cost of a missed default against a rejected good customer.
- **Split before you preprocess**: any imputation, scaling, or encoding statistics should come from the training set only.
- **Baseline first**: a regularized logistic regression is an honest benchmark to beat with boosted trees.
- **Use the bias/variance toolkit**: compare train and validation scores, draw a learning curve, and let that decide between "get more features" and "regularize more".
- **Error analysis**: inspect the defaults the model missed and look for a pattern in the features.

Revision questions

Q1. Why do we square the error in the cost function?

A. It makes errors positive, penalises big errors more, and gives a smooth convex cost with a simple derivative.

Q2. What does the learning rate do and how do you spot a bad one?

A. It scales the step size. If J rises or oscillates, α is too large; if J falls extremely slowly, it is too small.

Q3. Why do we scale features?

A. Different ranges make the cost contours elongated, so gradient descent zig-zags and converges slowly.

Q4. Why not use squared error for logistic regression?

A. Combined with the sigmoid it produces a non-convex cost with local minima; log loss is convex.

Q5. What does a large λ do?

A. It shrinks the weights toward zero, producing a smoother, simpler model that may underfit.

Q6. How do you tell high bias from high variance?

A. High bias: J_train is high compared with the baseline. High variance: J_cv is much larger than J_train.

Q7. Why is ReLU preferred in hidden layers?

A. It is cheap to compute and has no flat regions for positive inputs, so learning is faster than with sigmoid.

Q8. What is the purpose of the test set?

A. A final, untouched estimate of generalization performance; it must not guide any decision.

Q9. When is F1 better than accuracy?

A. When classes are heavily skewed, because accuracy can be high for a model that never finds the rare class.

Q10. What is information gain?

A. The reduction in entropy achieved by a split, weighted by the fraction of examples in each branch.

Q11. Why does a random forest help over one tree?

A. Averaging many trees trained on different samples reduces variance and the instability of single trees.

Q12. Why run K-means many times?

A. The result depends on initialisation and can be a local optimum; keep the run with the lowest cost J.

Q13. How does anomaly detection decide that a point is anomalous?

A. It fits Gaussian densities to normal data and flags points whose p(x) is below ε.

Q14. What is the cold-start problem?

A. A new user or item has no ratings, so collaborative filtering cannot make good predictions for it.

Q15. What does the Bellman equation say?

A. Q(s, a) equals the immediate reward plus γ times the best Q value reachable from the next state.